Asynchronous Communication with STOMP

What is STOMP?

STOMP (Simple Text Oriented Messaging Protocol) is a **lightweight, text-based protocol** that defines how clients communicate with message brokers over persistent connections (TCP or WebSocket).

👉 STOMP provides a **minimal, interoperable messaging standard**, independent of broker implementation.

Strengths

- Simple text format
- Easy debugging
- Language agnostic
- WebSocket-friendly

Limitations

- No native complex routing model
- No strong delivery guarantees by design
- Less efficient than binary protocols

Where STOMP is Used

Typical use cases

- Real-time web applications (via WebSocket)
- Microservices event-driven communication
- Notification systems
- Lightweight pub/sub architectures

Integration pattern

Client ⇄ Broker ⇄ Subscribers

Brokers Supporting STOMP

STOMP is supported natively or via plugins/extensions.

Broker	STOMP Version	Notes
RabbitMQ	1.2	Plugin-based STOMP support
ActiveMQ Artemis	1.1/1.2	Enterprise messaging platform
LavinMQ	1.2	High-performance broker
Solace PubSub+	1.2	Multi-protocol commercial broker
HornetQ	1.0	Legacy Java-based broker

STOMP over WebSocket

- Full-duplex communication
- Low latency
- Ideal for browsers

👉 STOMP over WebSocket enables real-time web messaging without polling.

Core Concept: Frames

STOMP communication is based on **frames**, which are structured text messages composed of:

1. Command
2. Headers
3. Body

Command

Defines the operation:

- CONNECT
- SEND
- SUBSCRIBE
- ACK
- DISCONNECT

Headers

Key-value metadata:

```
content-type: application/json
destination: /queue/orders
correlation-id: 12345
```

Body

Payload (JSON, XML, text)

Example: SEND Frame

```
SEND
destination:/queue/account-updates
content-type:application/json
priority:5
correlation-id:123e4567-e89b-12d3-a456-426614174000

{
  "accountId": "BA-1001-2025",
  "amount": 100.0
}
\0
```

👉 Frame terminator: null character (`\0`)

Destinations Model

STOMP uses **logical destinations** instead of broker-specific routing constructs.

Types

- `/queue/...` → point-to-point (competing consumers)
- `/topic/...` → publish/subscribe (broadcast)

Example

- `/queue/payments`
- `/topic/notifications`

Routing Semantics

👉 **STOMP does NOT define routing logic.**

STOMP is only responsible for defining **message format and communication semantics**, not how messages are routed inside the messaging system.

Layer	Responsibility
STOMP	Message format + commands (SEND, SUBSCRIBE, ACK)
Broker	Routing, delivery semantics, queues/topics, fanout, persistence

---

RabbitMQ (STOMP plugin)

- STOMP messages are translated into AMQP concepts
- `/queue/...` → queue (competing consumers)
- `/topic/...` → exchange-based routing (fanout / topic exchange)

👉 Actual routing depends on:

- exchange type
- binding configuration
- broker setup

ActiveMQ / Artemis

- Native STOMP support
- `/queue/...` → internal queue abstraction
- `/topic/...` → pub/sub topic with optional durable subscriptions

👉 Mapping is more direct because STOMP aligns with broker-native primitives.

Subscription Model

Clients subscribe to destinations:

```
SUBSCRIBE
id:sub-001
destination:/queue/money.deposit
ack:client-individual
\0
```

Acknowledgement Mechanisms

Modes

auto

- Broker automatically acknowledges delivery
- No client interaction required

client

- Client explicitly acknowledges messages
- ACK is cumulative up to message

client-individual

- Each message must be individually acknowledged
- Fine-grained control

Reliability Considerations

STOMP acknowledgements support:

- At-least-once delivery
- Backpressure handling
- Failure recovery

👉 However, exactly-once delivery is NOT guaranteed by STOMP itself.

Resources